Eventually Consistent Access Control with Deterministic Deny-Wins Replay for Multi-Stakeholder Offline Systems

Ulf Bodin, Eric Chiquito, Johan Kristiansson, Aditya Sissodiya
Lulea University of Technology, Sweden

Abstract—Multi-stakeholder systems need access control that works offline, supports independent issuers, and converges to a deterministic outcome after partitions. Existing approaches—centralized IAM, token-based schemes, pure CRDTs, and blockchains—fail to satisfy these requirements simultaneously. We present ECAC, an eventually consistent access control model that combines a causal event DAG with deterministic deny-wins replay. ECAC uses W3C Verifiable Credentials with in-band status lists for offline revocation, and supports per-tag encryption with key rotation. By treating policy and trust changes as first-class events in the same log as data, ECAC enables authorization decisions to be replayed deterministically across replicas.
We evaluate a Rust prototype using deterministic synthetic workloads and policy-heavy CBOR logs. In randomized delivery tests (100 permutations of a 10k-op log), all replicas converged to an identical state digest. Deny-wins revocation correctly skipped all post-revoke writes (500/500), and multi-issuer conflicts resolved deterministically. Replay time scales linearly ($t = 0.019n + 120$ ms, $R^2 = 0.998$); throughput reaches 45k ops/s for linear chains, and checkpoints yield 4–10$\times$ speedup. Offline partition merges invalidate post-revocation operations after reconciliation, and audit-chain tampering is detected. Confidentiality overhead is modest ($\sim$6.5%). These results demonstrate that ECAC provides deterministic, auditable authorization for offline multi-stakeholder environments without blockchain consensus, while maintaining practical performance.

Index Terms—Access control, revocation, verifiable credentials, CRDTs, deterministic replay, audit, offline systems, industrial IoT, data spaces.

1 Introduction

Access control in multi-stakeholder environments faces a fundamental tension: devices and services must continue operating when disconnected from central infrastructure, yet security-critical decisions—especially credential revocation—must eventually be enforced consistently across all parties. This tension is particularly acute in industrial IoT [1], [2], healthcare [3], and cross-organizational data sharing [4], [5], where equipment changes hands between manufacturers, operators, and service providers throughout its lifecycle.

Consider a connected industrial device produced by one company, deployed by a client organization, and later maintained by a third-party contractor. Each stakeholder needs to grant or revoke access permissions as ownership and trust relationships evolve. The device may operate in environments with intermittent connectivity [6], [7]—factory floors with spotty wireless coverage, remote installations, or air-gapped networks for security compliance. When the device reconnects, all parties must agree on the same authorization state, including which credentials were valid at what times.

Existing approaches fail to address this scenario adequately:

Centralized IAM systems assume continuous connectivity to a policy decision point (PDP). When the network is unavailable, authorization fails or falls back to cached decisions that may be stale [8]. Jacob and Hartenstein [9] formalize this limitation, showing that traditional access control models cannot provide meaningful guarantees under partition without additional mechanisms.

Token-based schemes (OAuth, JWT) enable offline validation of bearer tokens, but revocation remains problematic. Short-lived tokens require frequent renewal, which fails offline. Long-lived tokens create revocation windows during which compromised credentials remain valid [10]. Certificate revocation lists (CRLs) and OCSP introduce their own connectivity dependencies.

Blockchain-based access control has attracted significant attention [11]–[13] for its tamper-evident properties. However, requiring on-chain consensus for every policy change imposes latency incompatible with real-time operation and availability requirements during partition [14]. Multi-authority blockchain schemes [15], [16] address trust distribution but inherit the consensus bottleneck.

CRDT-based collaboration systems provide eventual consistency for replicated data [17], but lack built-in mechanisms for access control, credential verification, or revocation semantics. Naively applying CRDTs to authorization state can create security violations where conflicting grants and revokes merge unpredictably.

The gap is clear: no existing approach simultaneously provides offline operation, deterministic convergence, fine-grained revocation, multi-authority credentials, and tamper-evident audit—properties that are jointly required for secure multi-stakeholder collaboration.

This paper presents ECAC (Eventually Consistent Access

Control), a model that addresses this gap. ECAC combines four key elements:

1) A *causal event DAG* that records all operations (data writes, grants, revokes) as signed, hash-linked events, enabling deterministic replay regardless of delivery order.
2) *Deny-wins replay semantics* that guarantee revoked credentials have no effect on final state—even if revocation arrives after the operations it invalidates.
3) *W3C Verifiable Credentials* with embedded status lists for offline-verifiable, multi-authority authorization that requires no central registry.
4) *Per-tag encryption with key rotation* for confidentiality and forward secrecy, ensuring revoked principals cannot read data created after revocation.

Unlike blockchain approaches, ECAC requires no global consensus—replicas operate independently during partition and converge deterministically upon reconnection. Unlike pure CRDTs, ECAC integrates policy enforcement directly into the replay mechanism, ensuring that unauthorized operations are excluded from final state by construction.

Problem statement. How can multiple independent authorities issue and revoke fine-grained permissions for shared resources that operate offline for extended periods, while ensuring that all parties eventually converge on the same authorized state and can verify a tamper-evident history of access decisions?

1.1 Properties and Design Drivers (P1–P8)

The setting above imposes constraints that single-authority, always-online designs cannot meet: devices must work while disconnected, multiple organizations can grant or revoke independently, and all parties need a verifiable history of what happened.

From these conditions—and the tensions they create (availability vs. auditability; autonomy vs. trust; revocation timeliness vs. partitions; privacy vs. replication; integrity vs. storage growth)—we derive eight properties that any viable approach must satisfy:

P1 Offline availability. Authorization decisions must be enforceable without contacting a central PDP or ledger; disconnected nodes operate using locally verifiable evidence.

P2 Deterministic convergence. Given the same set of policy and credential events, all parties must compute the same final authorization state regardless of arrival order.

P3 Fine-grained revocation. Rights can be rescinded precisely (per subject, role, scope) with bounded windows of residual access under partitions, and without collateral effects.

P4 Tamper-evident audit. The history of grants, revokes, and decisions forms an append-only, verifiable trace that cannot be altered or erased without detection.

P5 Multi-authority credentials. Independent issuers can grant or revoke; verifiers check authenticity and status offline (e.g., via verifiable credentials and status lists).

P6 Bounded trust / Byzantine exposure. No single actor's compromise should subvert global policy; trust in any authority is scoped (by time, scope, or quorum).

P7 Confidentiality. Policy and data disclosures are minimized; unauthorized parties see redacted or encrypted content, and post-revocation unreadability is enforced.

P8 Scalability and compaction. Throughput, replay, and storage remain practical as events and participants grow; history can be checkpointed/compacted without losing verifiability.

These requirements are strict: they exclude designs assuming permanent connectivity, a single administrative root, or global consensus for every change. Section 2 evaluates mainstream approaches against P1–P8, making the gaps explicit and motivating the ECAC design choices that follow.

2 Gap Analysis

To position ECAC and motivate its design, we evaluate four classes of access control systems against P1–P8: centralized IAM, token-based approaches, pure CRDT-based systems, and blockchain-based access control. These categories capture the dominant paradigms in practice and research.

We apply each property as a binary or graded criterion based on published architectures and known failure modes. Tables 1–4 summarize results, highlighting where assumptions about connectivity, trust, or consensus conflict with offline multi-stakeholder requirements.

The pattern is consistent: each class satisfies a subset of properties but fails on the combination required for offline multi-authority collaboration. This gap motivates ECAC's hybrid approach, combining deterministic replay (P2), in-band policy and trust events (P3, P5, P6), and audit integrity (P4) without requiring always-online consensus (P1) or sacrificing confidentiality and scalability (P7, P8).

The gap analysis shows that no existing paradigm satisfies all eight properties. Centralized IAM breaks offline availability and multi-authority support; token systems struggle with revocation and auditability; CRDT-only designs provide convergence but lack policy integrity and bounded trust; blockchain-based systems trade availability and confidentiality for tamper-evidence. ECAC addresses this gap by combining deterministic replay and CRDT convergence (P1, P2) with in-band verifiable policy and trust events (P3, P5, P6) and audit integrity (P4), while retaining practical performance and optional confidentiality (P7, P8). The following section presents the ECAC model.

3 ECAC Model

ECAC is an Eventually Consistent Access Control model designed for offline-first operation with rigorous security and auditability. This section presents the conceptual contributions rather than implementation details.

ECAC combines four key elements: (1) a causal event log with deterministic deny-wins replay, (2) verifiable credentials for time-bounded authorization epochs, (3) a hybrid ABAC/RBAC policy model with deny-overrides semantics, and (4) per-tag encryption with forward secrecy.

We outline formal guarantees for convergence, safety, determinism, audit integrity, and forward secrecy. Figure 1 presents a high-level view of the system's four-phase lifecycle: ingestion of signed operations, credential validation,

TABLE 1
Evaluation of Centralized IAM Systems Against ECAC Requirements (P1–P8) Centralized IAM systems excel at maintaining a consistent policy view in real time under ideal network conditions, but they fail to support decentralized, partition-resilient operation. They demand complete trust in a single party, leaving requirements P1, P3, P5, and P8— and partially P4 and P7—unmet in dynamic, multi-stakeholder environments [9], [11], [1].

Properties	Observation and Analysis
P1 Offline Availability	Traditional enterprise IAM or directory services rely on a single authoritative server (or tightly coordinated cluster) to make and enforce access decisions. This architecture fundamentally requires online connectivity and a unified trust domain. When the central authority or network link is unreachable, users cannot authenticate or authorize, and local resources are unable to evaluate policy autonomously. Consequently, centralized IAM systems violate offline availability [9], [11], [1].
P2 Deterministic Convergence	The strong consistency of a central database trivially avoids divergent states, providing a linearizable view of policy by design. While this sidesteps the need for deterministic convergence logic, it does so at the expense of availability and partition tolerance: any network partition isolates outlying sites from policy updates, halting authorization outside the central domain [9], [11].
P3 Timely Revocation Across Partitions	Revocation is not achievable under network partitions. If connectivity to the central authority is lost, revocation events (such as disabling a user or credential) cannot reach disconnected segments, leaving a window during which stale permissions continue to be honored. This delay in propagation undermines the timeliness of revocation in distributed or intermittently connected environments [9], [14], [15].
P4 Tamper-Evident Auditability	Audit logs in centralized IAM systems are maintained under the same administrative control as the access decisions themselves. While these logs can facilitate oversight, they are not tamper-evident without additional safeguards. A privileged operator or attacker with access to the logging infrastructure can alter or delete records undetected, as there is no cryptographic or structural mechanism ensuring immutability [11], [18], [19].
P5 Multi-Authority Credentialing	Centralized IAM designs consolidate all identity and authorization control within a single administrative realm, which conflicts with the notion of multi-authority credentialing. All entities—users, services, and organizations—must defer to the same root of trust, preventing multiple issuers from participating independently or establishing decentralized trust relationships [16], [15], [9].
P6 Confidentiality and Forward Secrecy	Centralized IAMs depend on transport-layer encryption (e.g., TLS) to protect credentials in transit and database access controls to secure data at rest. However, the central repository constitutes a high-value target; compromise of this system can expose large volumes of sensitive credential data. Furthermore, if long-term keys are compromised, past communications may be decrypted, since forward secrecy is typically enforced only at the session layer and not by the IAM system itself [8], [20], [9].
P7 Scalability and Compaction	Although central services can scale vertically or through clustered replicas, a single logical authority often becomes a performance bottleneck in large, geographically distributed deployments. Caching and replication techniques may improve throughput but introduce additional complexity and reduce consistency guarantees, as cached nodes may serve stale policy data and delay global updates [9], [1], [6].
P8 Bounded Trust and Byzantine Resilience	Centralized IAM inherently concentrates trust in a single authority. The entire system's security and availability depend on the correct and honest functioning of that authority. There is no resilience if it is compromised or behaves maliciously. Such a model provides no protection against insider threats or Byzantine failures [9], [11], [10].

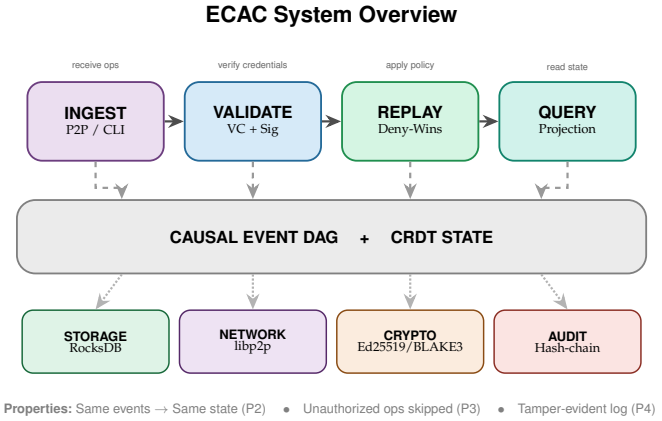

Fig. 1. ECAC system overview showing the four-phase lifecycle (Ingest, Validate, Replay, Query), central causal DAG with CRDT state, and supporting infrastructure.

policy-gated replay, and state projection. Subsequent figures detail each component.

3.1 Event Log and Deterministic Deny-Wins Replay

ECAC maintains an append-only event log structured as a causal DAG, where each event points to its causal predecessors. The DAG encodes happens-before relations, and system state is derived by deterministically sorting and replaying events. A fixed traversal order ensures every replica produces the same sequence.

Concurrent events—those without causal order—are resolved using a composite timestamp: a Hybrid Logical Clock (HLC) value plus a unique operation ID. The HLC combines physical and logical time to preserve causality; when timestamps match, op IDs break ties deterministically. Replay thus becomes a pure function of the event set, guaranteeing identical state across replicas.

Figure 2 illustrates the replay-and-deny pipeline.

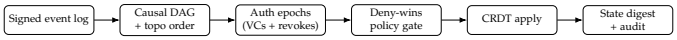

Fig. 2. Deterministic replay with deny-wins filtering.

The event DAG acts as a convergent CRDT: merging logs from partitions is a set union of nodes and edges—commutative, associative, and idempotent—ensuring eventual consistency.

TABLE 2
Evaluation of Token-Based Approaches (JWT, OAuth2, X.509) Against ECAC Requirements (P1–P8) Token-based approaches improve scalability and enable partial decentralization of access decisions, fulfilling P1 and P7 to a significant degree. However, they struggle with revocation timeliness (P3), lack built-in tamper-evident auditability (P4), and remain dependent on centralized trust anchors (P5, P8). These shortcomings limit their suitability for environments requiring strong partition-tolerance, dynamic trust, and verifiable global policy correctness.

Properties	Observation and Analysis
P1 Offline Availability	Token-driven access control schemes issue cryptographically signed tokens or certificates that embody a user's permissions, which can be validated locally without contacting a central authority. As long as a resource node possesses the token and the issuer's public key, it can autonomously authenticate and authorize requests. This design provides better offline availability than centralized IAM, enabling authorization even when disconnected from the issuer [21], [13], [22].
P2 Deterministic Convergence	While token verification is stateless and deterministic at each node, token-based systems lack a global notion of convergent policy state. Tokens are issued and revoked asynchronously; once issued, their validity is independent of real-time synchronization. Thus, while each node deterministically verifies signatures, the overall system may still diverge on who is currently authorized during partitions [9], [14], [9].
P3 Timely Revocation Across Partitions	Revocation remains a persistent weakness. Once a token or certificate is issued, rescinding it before its expiration is non-trivial, especially when clients or servers are offline. Techniques such as Certificate Revocation Lists (CRLs), Online Certificate Status Protocol (OCSP) checks, or short-lived tokens can reduce risk, but none guarantee that revocations propagate instantly across partitions. A user isolated from the network can continue using a still-valid token until it expires, creating a window where stale permissions remain effective [14], [15], [23].
P4 Tamper-Evident Auditability	Each token is cryptographically tamper-resistant—it cannot be altered without invalidating its signature—but token systems do not inherently log their use in a tamper-evident manner. There is no unified append-only record of where and when tokens are presented. Accountability is retrospective: auditors must gather event logs from every resource server, which may be incomplete or falsifiable if not independently secured [11], [18], [10]. This limits audit integrity and forensic traceability.
P5 Multi-Authority Credentialing	Token systems typically operate under a centralized or federated trust model rather than true multi-authority control. Users and services depend on one or a few token issuers (identity providers or certificate authorities). Federations of multiple issuers are possible, such as trusting several root CAs or enabling cross-domain identity, but these arrangements rely on static trust agreements and manual configuration. The model lacks mechanisms for dynamic, decentralized trust establishment among peers [16], [15], [9].
P6 Confidentiality and Forward Secrecy	Confidentiality in token systems is largely external to the token. Many tokens, such as JWTs, include plaintext claims that can reveal sensitive information if intercepted. They depend on transport-layer encryption (HTTPS/TLS) for confidentiality in transit. Forward secrecy is not intrinsic: if an attacker steals a valid token from memory or cache, it remains usable until expiration. Tokens function as bearer credentials and do not automatically expire upon key rotation or user disconnection unless re-validation is enforced by additional protocols [8], [20], [9].
P7 Scalability and Compaction	Stateless verification significantly improves scalability. Because tokens are self-contained, each node can verify them in constant time without consulting a global database, distributing authentication load and reducing contention. This property allows token-based architectures to scale horizontally with minimal coordination overhead. However, scalability is achieved at the cost of real-time revocation and global policy coherence [13], [21], [9].
P8 Bounded Trust and Byzantine Resilience	The trust model in token-based systems remains centralized. Token validation depends entirely on trusting the issuer's private key and correct behavior. A malicious or compromised issuer could mint unauthorized tokens, and there is no built-in mechanism to detect or recover from such misbehavior short of revoking trust in that issuer altogether. While federations spread risk across multiple issuers, they still rely on pre-established static trust roots rather than active Byzantine fault tolerance [9], [11], [10].

Figure 3 details the operation structure and DAG mechanics. Each operation carries a signed header with parent links, HLC timestamp, author key, and typed payload. The DAG maintains two staging areas: activated nodes (all parents present) feed the deterministic replay, while pending nodes wait for missing parents before activation.

Replay follows a deny-wins rule: apply if authorized at event time, else skip. Each event carries its logical timestamp and is applied only if the actor held the required permission at that moment. Unauthorized events become no-ops. This timestamp-ordered authorization ensures that policy-invalid actions have no effect on final state. The combination of causal order, deterministic tie-breaks, and deny-wins replay guarantees all replicas omit the same unauthorized operations, yielding a uniform, policy-consistent result (P2).

3.2 Authorization Epochs via Verifiable Credentials and Status Lists

ECAC manages permissions using verifiable, time-bounded credentials. It adopts the W3C Verifiable Credentials (VC) format for signed authorization tokens asserting a principal's rights over a specific interval (validFrom–validUntil). Credentials are tamper-evident and self-verifying; replicas independently confirm authenticity and validity.

Revocations use standard VC status lists—cryptographically signed bitstrings indicating whether each credential ID is revoked. A valid credential defines an authorization epoch; expiry or revocation ends it.

Every grant and revoke event is recorded in the event log as part of a TrustView, containing the issuer's public key and a digest of the current revocation registry. Embedding this data removes dependence on a central authority: an offline node can verify at any timestamp that a user held a valid, non-revoked credential. The log itself forms a distributed PKI, replicating issuer keys and revocation data across

TABLE 3
Evaluation of Pure CRDT-Based Systems Against ECAC Requirements (P1–P8). Pure CRDT-based access control fulfills key properties of availability and deterministic convergence (P1, P2) and supports multi-authority participation (P5). However, it performs poorly on timely revocation (P3), tamper-evident auditability (P4), confidentiality (P6), and Byzantine resilience (P8). Scalability (P7) is technically achievable but constrained by metadata growth. In essence, CRDT-based AC provides a decentralized framework that aligns with ECAC's goals but lacks the necessary audit, revocation, and trust guarantees required for critical or adversarial environments [9], [5], [24].

Properties	Observation and Analysis
P1 Offline Availability	Conflict-free replicated data types (CRDTs) have been proposed as a foundation for fully distributed access control, eliminating any need for global consensus or a central coordinator. In such systems, policy information—user roles, permissions, and revocations—is represented as a CRDT and replicated across all nodes. Each participant can independently modify local policy state (grant or revoke access) while offline, satisfying the offline availability requirement. Nodes synchronize opportunistically when connectivity is restored, ensuring progress without dependence on a central service [9], [24], [5].
P2 Deterministic Convergence	CRDTs are designed so that all replicas converge toward a consistent state once they exchange updates, regardless of message order or delay. This guarantees deterministic convergence: every node that eventually observes the same set of operations arrives at an identical final policy state. This property directly aligns with ECAC's goal of deterministic replay, providing a theoretical basis for "act now, reconcile later." [9], [24], [5]
P3 Timely Revocation Across Partitions	Revocation under network partitioning remains a key limitation. While a revocation event eventually propagates and takes effect globally, during disconnection it cannot immediately restrict access at remote replicas. If a user's clearance is revoked on node A, a disconnected node B may still consider that user authorized until synchronization occurs. This unavoidable "window of inconsistency" is unacceptable in high-security contexts and difficult to bound without adding coordination mechanisms or timeout-based safeguards, which violate the CRDT model's coordination-free premise [9], [5], [25].
P4 Tamper-Evident Auditability	Achieving tamper-evident auditability in a pure CRDT framework is non-trivial. The model focuses on convergence of final state, not on maintaining an immutable event history. A malicious participant could withhold, replay, or selectively share updates, effectively rewriting history. Digital signatures on CRDT operations can ensure that only authorized parties issue valid updates and can detect some forgery, but they do not provide an immutable audit trail. Without a global append-only ledger, actions are not incontrovertibly auditable; compromised nodes could falsify local histories, and reconstructing an authoritative log requires cross-verification among replicas [9], [11], [18].
P5 Multi-Authority Credentialing	CRDT-based systems naturally support a multi-authority model. Every replica—representing a stakeholder organization—acts as a peer capable of issuing updates to the access control state. Policies emerge from the collective union of these updates, with no central master. This decentralization aligns with multi-stakeholder governance needs, allowing independent authorities to coexist without a single root of trust [9], [24], [16].
P6 Confidentiality and Forward Secrecy	Confidentiality poses a major challenge. Because CRDTs replicate the entire access control state across all participants, sensitive policy information (e.g., user attributes, roles, or access histories) becomes visible to every node. Unless encrypted in a merge-compatible manner—an unsolved research problem—replication conflicts with confidentiality. Achieving forward secrecy would require frequent key rotations or ephemeral encryption per update, adding complexity and coordination overhead that undermine CRDT simplicity [20], [26], [8].
P7 Scalability and Compaction	Scalability in CRDT-based systems is mixed. On one hand, CRDTs scale horizontally by allowing many nodes to operate and update concurrently without central bottlenecks, and they tolerate intermittent connectivity gracefully. On the other hand, metadata growth—such as tombstones, per-replica version vectors, and retained operation logs—can become significant over time. Without compaction or state pruning, memory and synchronization costs increase unboundedly. However, compaction (e.g., snapshotting or garbage collection) reintroduces coordination and potential trust assumptions to avoid data loss, compromising pure decentralization [9], [5], [24].
P8 Bounded Trust and Byzantine Resilience	CRDT systems implicitly assume all replicas behave honestly and eventually exchange updates. They lack intrinsic Byzantine fault tolerance: a malicious node could inject fabricated updates, withhold revocations, or refuse to relay others' updates. Detecting or correcting such behavior is difficult without external verification. Some resilience can be added using threshold signatures or partial agreement protocols, but these additions blur the boundary between CRDTs and consensus systems, undermining the "pure" CRDT model [9], [25], [10].

nodes for local, offline validation.

During replay, the system checks for a valid credential covering each event's timestamp. If found, the operation applies; otherwise, it is skipped. These time-scoped credentials enforce temporal access control: users cannot act outside active grant intervals, and all replicas reach identical decisions since they verify identical evidence.

ECAC's authorization model thus uses decentralized, verifiable credentials to issue and revoke rights securely, allowing any node to prove *who* was allowed to do *what* and *when*—without contacting any live identity service.

Figure 4 illustrates the in-band trust assembly and VC verification flow. Trust operations (IssuerKey, IssuerKeyRevoke, StatusListChunk) build the TrustView incrementally during replay. When a VC needs verification, the system parses the JWT, looks up the issuer key in TrustView, verifies the Ed25519 signature, checks status list revocation, and extracts claims.

Figure 5 illustrates how grants, revocations, and operations interact over time during replay.

TABLE 4
Evaluation of Blockchain-Based Access Control Systems Against ECAC Requirements (P1–P8) Blockchain-based access control excels in ensuring global consistency, tamper-evident auditing, distributed authority, and bounded trust (P2, P4, P5, P8). Yet it performs poorly in scenarios requiring offline operation (P1), immediate revocation under partition (P3), and privacy preservation (P6), and it faces ongoing scalability and storage constraints (P7). While offering strong integrity and decentralization, these systems trade away availability, confidentiality, and efficiency, making them ill-suited for offline or latency-sensitive environments [11], [18], [19].

Properties	Observation and Analysis
P1 Offline Availability	Blockchain-based access control systems depend on continuous connectivity for correct operation. Every grant, revoke, or audit record must be submitted to and confirmed by the global ledger. Purely offline availability is incompatible with this requirement: a node disconnected from the network cannot view or record new transactions until it reconnects and synchronizes with the latest chain state. While cached state can support temporary decisions, those decisions may rely on stale information. Consequently, blockchain AC prioritizes consistency and auditability over availability, opposite to the CRDT philosophy [6], [11], [9].
P2 Deterministic Convergence	Blockchains guarantee deterministic convergence by enforcing a total order of transactions through consensus. Once a transaction is confirmed, all nodes converge on an identical, globally agreed-upon state. This property provides deterministic convergence in a stronger sense than eventual consistency—akin to total order broadcast. Every participant can independently recompute the same result from the ordered transaction history, eliminating ambiguity about policy state [27], [12], [11].
P3 Timely Revocation Across Partitions	Blockchain propagation delays and partition tolerance constraints limit revocation timeliness. In normal network conditions, revocation transactions spread quickly and become globally verifiable within seconds or minutes. However, under true partitions or prolonged outages, disconnected nodes cannot receive or apply new revocations. During these periods, revoked permissions remain active locally, leaving a window of exposure. Thus, "timely" revocation in blockchain AC holds only when the network functions normally; under offline conditions, it offers no better enforcement than other eventual consistency models [14], [15], [23].
P4 Tamper-Evident Auditability	Auditability is the strongest feature of blockchain-based systems. The immutable, append-only ledger ensures that once recorded, grants, revocations, and access events cannot be altered or deleted without detection. Every transaction is cryptographically linked to its predecessor, providing verifiable proof of history. Any participant can independently audit the sequence of access control operations, ensuring integrity and non-repudiation of actions [11], [18], [19].
P5 Multi-Authority Credentialing	Blockchains inherently support multi-authority governance. Multiple organizations can act as peers or validators on the ledger, each authorized to issue and verify credentials or policy updates. This shared source of truth eliminates reliance on a single authority. In consortium or public networks, control is distributed among many participants, enabling decentralized policy management while maintaining global consistency [10], [4], [23].
P6 Confidentiality and Forward Secrecy	Confidentiality remains a major limitation. Most blockchain implementations record policy data or credential hashes in a public or semi-public ledger visible to all participants. Without advanced cryptography—such as zero-knowledge proofs or encrypted fields readable only by authorized parties—sensitive information (e.g., who has access to which resource) may be exposed. Even if encrypted, forward secrecy is difficult to achieve: historical ciphertext remains permanently stored and could be decrypted in the future if keys leak or encryption algorithms are broken. The immutability that guarantees integrity simultaneously undermines long-term confidentiality [28], [29], [11].
P7 Scalability and Compaction	Scalability and compaction are persistent challenges. Consensus protocols impose latency and throughput limits; every policy update must be agreed upon by the network, significantly reducing transaction rates relative to centralized systems. High-frequency events—such as per-device or per-session access logs—can quickly saturate blockchains. Furthermore, the ledger grows continuously as every transaction is retained for auditability. Although pruning and state checkpointing techniques exist, the need for verifiable history prevents full compaction. Permissioned or layer-2 solutions can improve throughput but typically reintroduce partial trust or reduce decentralization [27], [19], [30].
P8 Bounded Trust and Byzantine Resilience	Blockchain consensus protocols eliminate dependence on any single trusted party and tolerate a bounded fraction of faulty or malicious nodes. Byzantine fault tolerance (BFT) ensures that, within defined thresholds, the system maintains consistent and correct state even if some participants behave dishonestly. This property satisfies the spirit of bounded trust: security is derived from distributed verification rather than absolute reliance on one administrator. However, BFT comes with computational overhead and reduced scalability as network size increases [27], [12], [10].

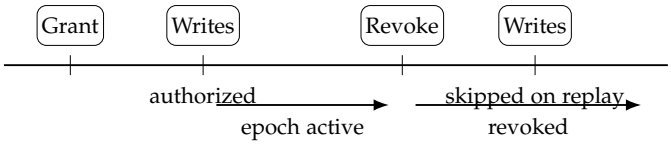

Fig. 5. Grant, revoke, and data operations over time; post-revocation writes are skipped during replay.

3.3 Policy Semantics (Cedar) and Deny-Overrides Alignment

ECAC builds on verifiable credentials with a rich policy layer inspired by AWS's Cedar language, an open-source ABAC framework. Policies may permit or forbid actions, with a default deny stance: if nothing explicitly permits, access is denied. ECAC adopts Cedar's deny-overrides (forbid-wins) semantics to extend the deny-wins rule across partitions.

If any matching policy forbids an operation, that denial overrides all permits. During replay, ECAC evaluates policies at the event's timestamp. If a forbid rule applies, the event is skipped; if only permits apply, it proceeds; if no rules apply, the default deny skips it. This mirrors XACML/Cedar's deny-overrides algorithm, ensuring predictable, safe-by-default access control.

ECAC unifies RBAC and ABAC: roles are attributes of principals (e.g., `role="admin"`), and policies can grant per-

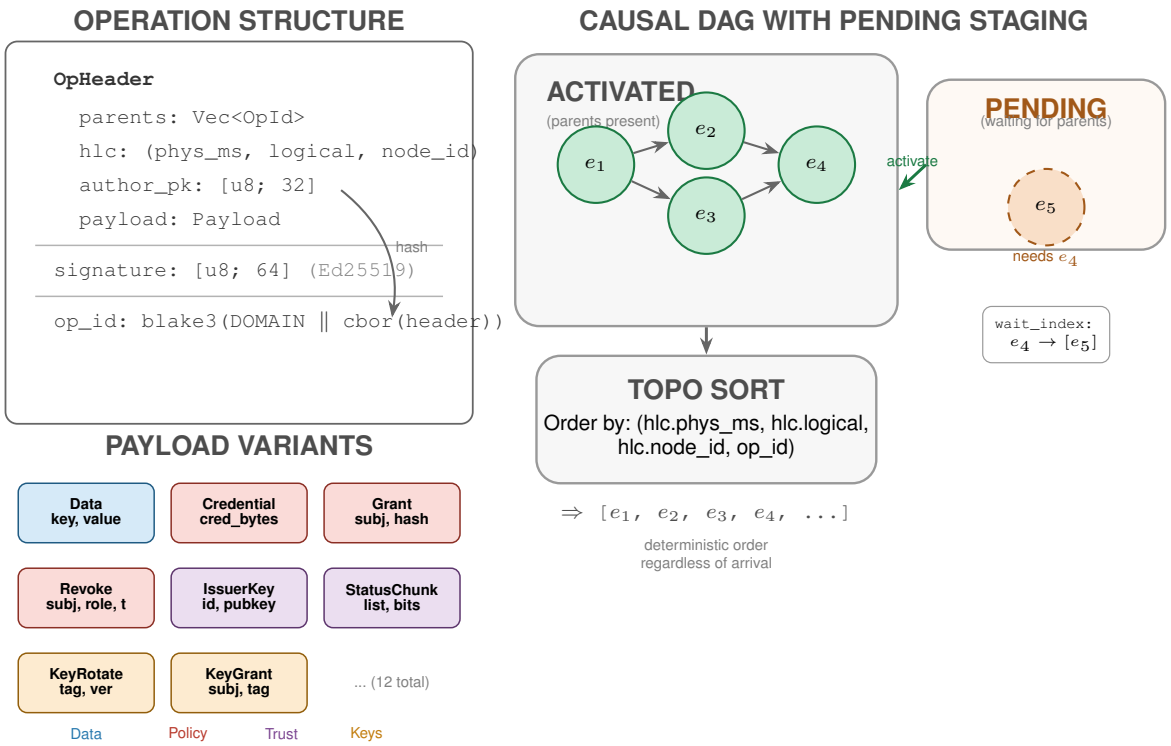

Fig. 3. Operation structure showing the OpHeader fields and 12 payload variants (left), and the causal DAG with pending-parent staging and deterministic topological sort (right).

missions directly or through roles. Cedar's entity-attribute model supports hierarchical roles and contextual attributes (department, clearance, object sensitivity, time-of-day), enabling fine-grained, context-aware rules alongside traditional role-based logic.

Policies reference system state—credentials and object metadata—and are evaluated deterministically. Identical inputs guarantee identical outcomes across replicas. The deny-overrides rule ensures partition-tolerant enforcement: a revocation on one node always takes precedence after logs merge, even if another node concurrently granted access. Deny-wins ensures that any revocation, once observed, deterministically overrides concurrent or later grants, closing the safety hole that partitions would otherwise create (P3).

Figure 6 shows the authorization epoch construction and deny-wins gate. Grant operations open epochs by recording subject, role, scope, and temporal bounds from the VC. Revoke operations close epochs by setting the end position. During replay, each DATA operation passes through the deny-wins gate, which checks epoch coverage, role permissions, scope intersection, and temporal validity before applying or skipping.

3.4 Confidentiality and Forward Secrecy

ECAC enforces data confidentiality through tagged encryption. Each event payload is labeled with an access tag (e.g., resource ID or classification) and encrypted so only principals with valid credentials for that tag can decrypt it. Each tag corresponds to a symmetric key shared with authorized users. Unauthorized users see only ciphertext but still replicate and verify the log. Encrypted events a user cannot decrypt are treated as no-ops during replay, preserving integrity and consistency.

To protect data after revocations, ECAC enforces forward secrecy via key rotation. When an authorization epoch ends, a new key is generated for that tag and distributed only to remaining authorized principals. Future events use the new key, making them unreadable to anyone holding only the old key. Old keys decrypt only content from their valid period; newly authorized users gain access only to data shared post-grant.

Key distribution and rotation are managed through the event log itself. A key rotation event records the new key encrypted to each authorized user's public key. During replay, authorized replicas retrieve the new key; unauthorized ones cannot. This integrates key management with causal history, ensuring consistent, verifiable key state across replicas.

All encrypted events remain hash-chained and signed, so log integrity and tamper detection are universal—even to users who cannot read contents. Unauthorized users compute a valid redacted subset of global state, maintaining structural consistency while upholding strict confidentiality and forward secrecy (P7).

Figure 7 illustrates the per-tag encryption and key rotation mechanisms. Data operations are encrypted using the tag's current symmetric key (XChaCha20-Poly1305). KeyRotate operations establish new key versions, and KeyGrant operations distribute keys encrypted to authorized subjects' public keys. The forward secrecy timeline shows how revoked principals cannot decrypt data created after key rotation.

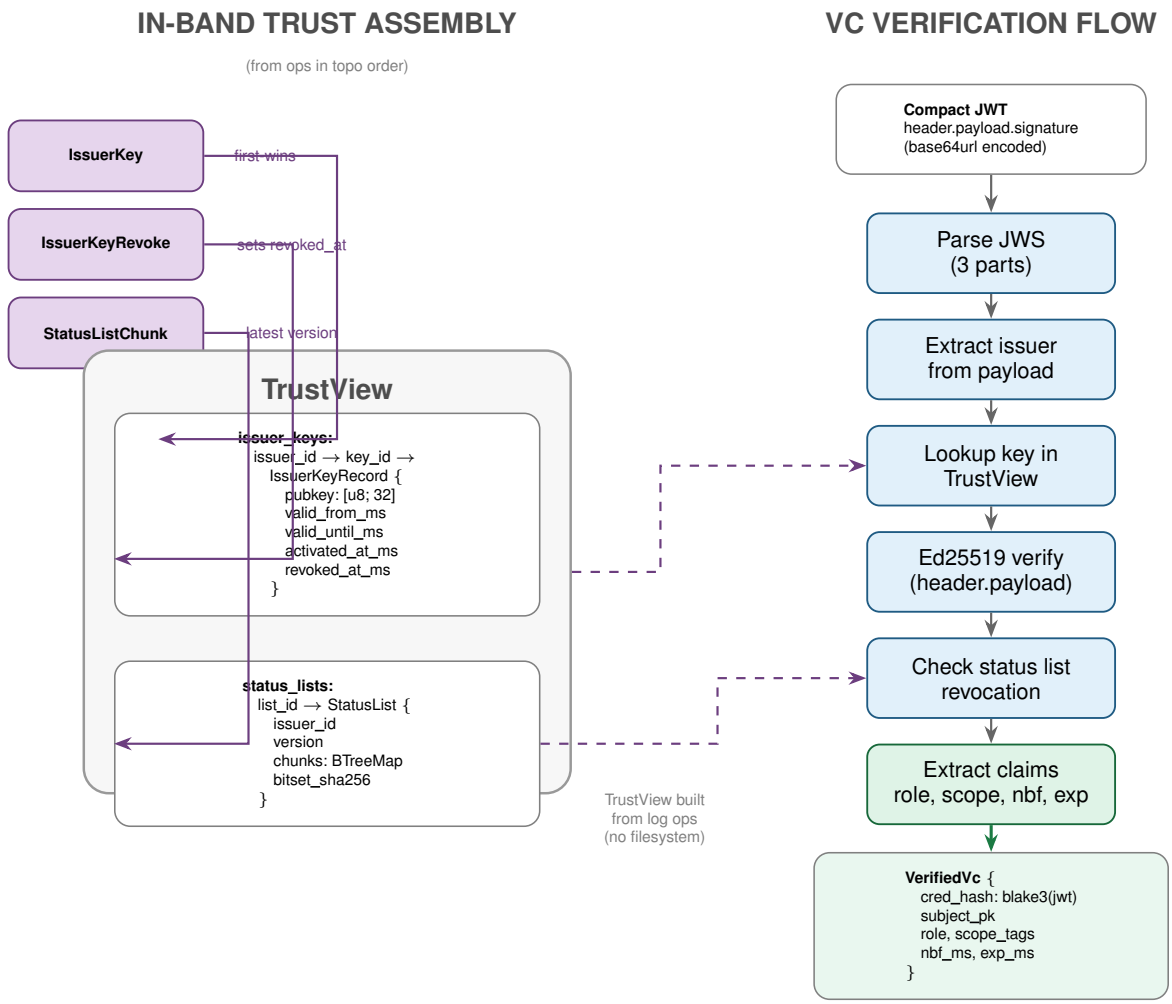

Fig. 4. In-band trust assembly from ops (left) and VC verification flow (right). TrustView is built entirely from log operations, requiring no external PKI.

3.5 Formal Guarantees (Proof Sketches)

We outline key formal properties of ECAC as proof sketches to demonstrate that the design meets its claims.

3.5.1 Convergence (Same events → same state)

- **Claim:** If two replicas process the same set of events, they reach an identical final state regardless of delivery order.
- **Justification:** The deterministic replay algorithm ensures convergence. Every event is applied in causal order with a globally-consistent tie-break, making application order a pure function of the input DAG. Merging logs from disconnected replicas is a DAG union (commutative and idempotent). All replicas sort and replay identically, analogous to deterministic state machine replication. ECAC thus satisfies Strong Eventual Consistency.

3.5.2 Policy Safety (No unauthorized effects in final state)

- **Claim:** In the final state, no operation affects the state if it was not authorized under policies in force at its timestamp.
- **Justification:** By construction, unauthorized events are skipped (deny-wins). Proof by contrapositive: if event e affected final state, it was applied, meaning it passed authorization at its timestamp—the principal had a valid credential and no forbid rule matched. Thus no unauthorized event contributes to state.

Formally, an inductive invariant holds: at any prefix of replay order, partial state results only from authorized operations. Initially true (empty state); if true up to e_i, then e_{i+1} either passes authorization (applied, invariant preserved) or fails (skipped, state unchanged). The final state reflects only authorized operations.

3.5.3 Determinism (Replay is a pure function of the DAG)

- **Claim:** Replay outcome is fully determined by the log itself, independent of message ordering, network delays, or nondeterministic choices.
- **Justification:** ECAC's replay is a deterministic function $F(\text{DAG}) \rightarrow$ state. The topological sort and application algorithm is fixed with no random or timing-based choices.

This determinism relies on:

1) A total order on concurrent events via timestamps and IDs—every tie breaks identically on all nodes.
2) Deterministic policy evaluation—authorization is a pure function of event, credential, and policy inputs.

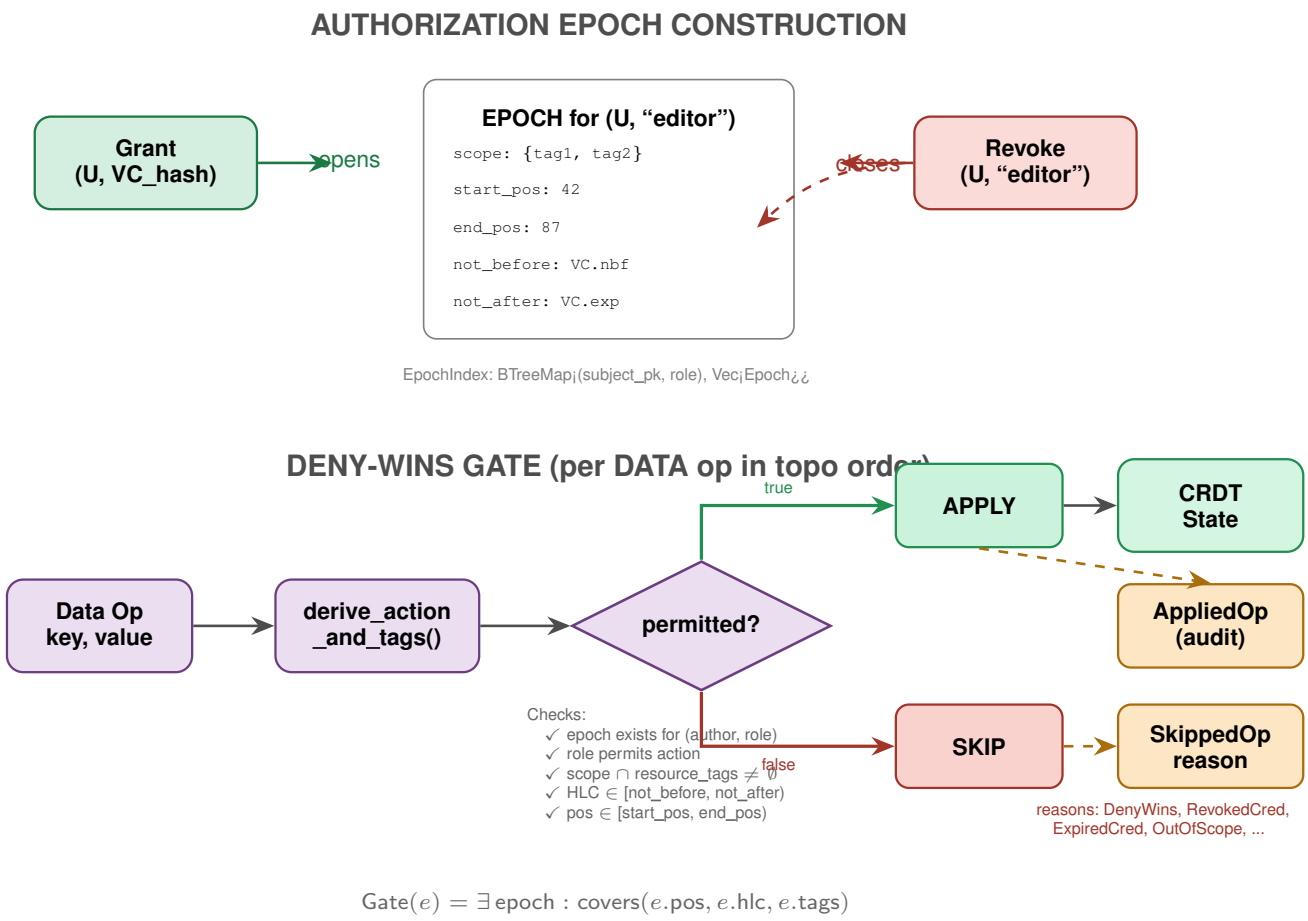

Fig. 6. Authorization epoch construction via Grant/Revoke (top) and the deny-wins gate that filters each DATA operation (bottom). Applied and skipped operations generate audit events.

3) Deterministic cryptographic primitives—hash, signature verify, and decrypt produce consistent outcomes.
4) No wall-clock dependence—all temporal decisions use event timestamps.

The entire replay is self-contained and deterministic, enabling consistent audits and verifiable outcomes.

3.5.4 *Audit Integrity (Tamper-evident chain)*

- **Claim:** The event log is a tamper-evident audit trail. Any alteration (add, remove, or change events) is detectable.
- **Justification:** Each event includes a cryptographic hash linking it to predecessors, creating a Merkle-DAG commitment. Tampering breaks hash verification. Events are signed by originators for origin authenticity.

The log is append-only; post-hoc modifications are evident. Since every applied operation appears as an event, and skipped events have no effect, the log provides a complete semantic trace of system behavior. The state after replay equals the result of applying all non-skipped events in order. Auditors can verify that no unauthorized operation occurred by checking policy decisions for each event. ECAC provides an immutable ledger with strong integrity guarantees; discrepancies between replicas are detected upon synchronization when hashes fail to match (P4).

3.5.5 *Forward Secrecy (Post-rotation unreadability)*

- **Claim:** A principal revoked at time t cannot decrypt data created after t, even with copies of prior keys. Compromise of a past key does not expose future events.
- **Justification:** Let $K_1, K_2, \ldots$ be the sequence of keys for a tag, with rotations at times $T_1, T_2, \ldots$. A principal revoked at T_r has keys $K_1 \ldots K_{r-1}$ but not K_r or later. Events after T_r use keys the revoked principal cannot obtain.

This is forward secrecy applied to access control: history partitions into epochs demarcated by rotations, and read access is limited to authorized epochs. New keys are distributed by encrypting to each authorized user's public key—an operation the revoked user cannot access. ECAC also provides backward secrecy: newly authorized users cannot decrypt past records unless explicitly re-shared. These properties uphold confidentiality over time, aligning with least-privilege principles.

3.6 ECAC Model and Guarantees

3.6.0.1 Domains and artifacts.: Let Subj be subjects (principals), Res resources, Act actions (scopes), and Attr attribute valuations. Time is totally ordered as $(\mathbb{T}, \leq)$; we write intervals $[t_1, t_2)$ with $t_1 < t_2$. Let H be a collision-resistant hash and $\mathsf{Sig}_k(\cdot)$ a secure signature scheme.

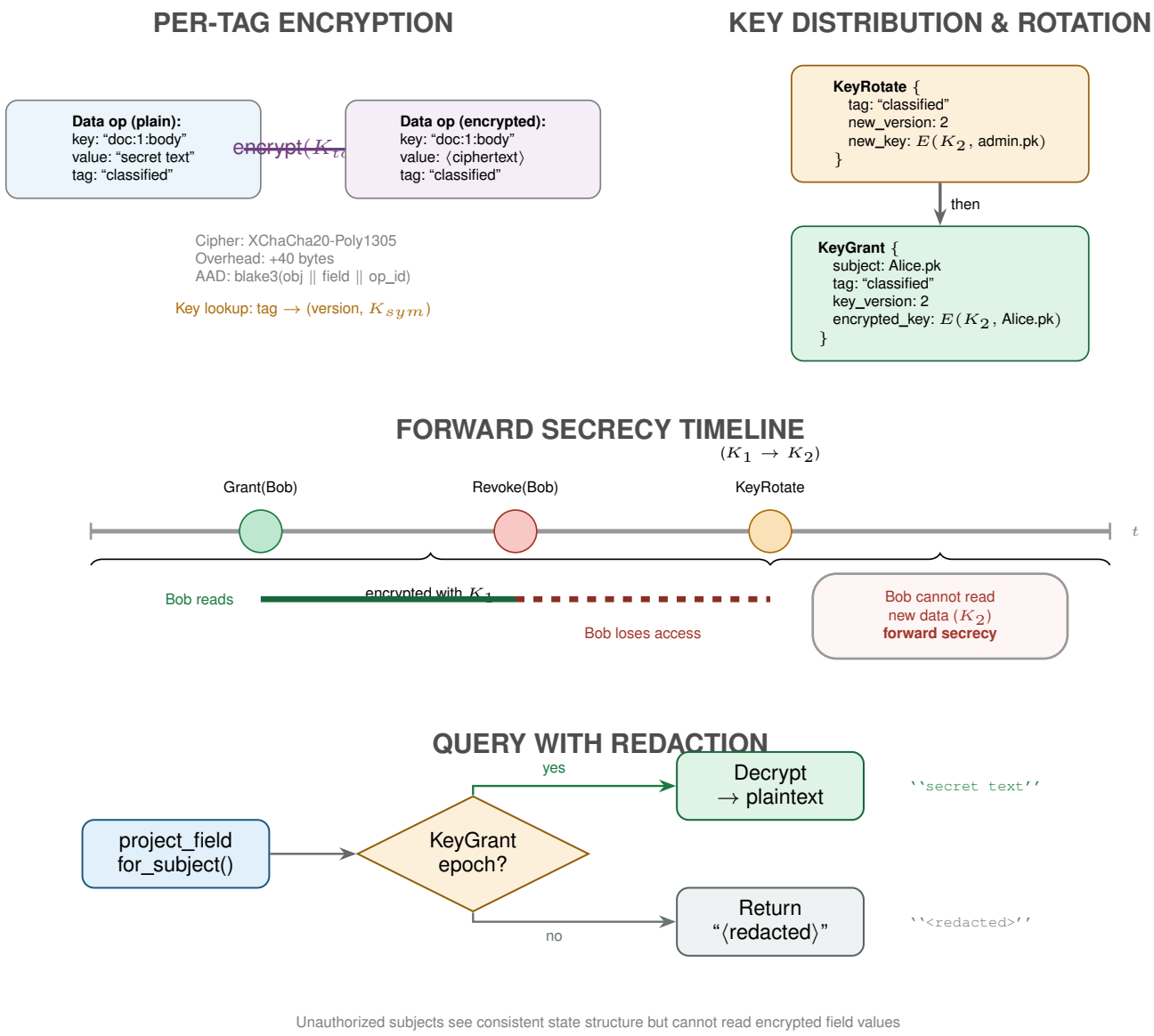

Fig. 7. Per-tag encryption (top left), key distribution and rotation (top right), forward secrecy timeline (middle), and query-time redaction for unauthorized subjects (bottom).

Verifiable credentials and trust. A verifiable credential (VC) is a tuple

$$c = \langle s,\ \alpha,\ V,\ \mathrm{nb},\ \mathrm{na},\ \sigma_I \rangle$$

where $s \in \mathsf{Subj}$, $\alpha \in \mathsf{Attr}$ (roles/scopes), V issuer identifier, and $\mathrm{nb}, \mathrm{na} \in \mathbb{T}$ are not-before/after bounds. σ_I is a signature under issuer key K_V. A status list is a succinct map $\mathsf{SL}_V : \mathrm{id}(c) \mapsto \{\mathsf{good}, \mathsf{revoked}\}$ with short TTL. A *TrustView* event carries (i) issuer keys $\{K_V\}$ and (ii) digests of $\{\mathsf{SL}_V\}$, recorded in-band so replicas can validate VCs offline.

Events and log. Events are partitioned as

$$\mathsf{Ev} = \mathsf{DATA} \uplus \mathsf{GRANT} \uplus \mathsf{REVOKE} \uplus \mathsf{TRUST}.$$

Every event $e \in \mathsf{Ev}$ has header $h(e)$ including domain fields, causal parents $\mathsf{par}(e) \subseteq \mathsf{Ev}$, a hybrid logical clock $\mathrm{hlc}(e) \in \mathbb{T}$, and a producer signature σ_e. The identifier is

$$\mathsf{op_id}(e) = \mathsf{H}(\mathrm{domain} \,\|\, \mathrm{CBOR}(h(e))).$$

The *signed, hash-linked op log* is an append-only bag of events with edges $e' \rightarrow e$ for $e' \in \mathsf{par}(e)$. The induced graph $G = (\mathsf{Ev}, E)$ must be a DAG; this causal DAG defines the partial order $\prec$ as the transitive closure of E.

Deterministic order. Define a total order $\lhd$ over Ev that *extends* causality:

$$e_1 \lhd e_2 \text{ if } (e_1 \prec e_2) \text{ or } \Big(e_1 \not\prec e_2 \wedge e_2 \not\prec e_1 \wedge (\mathrm{hlc}(e_1), \mathsf{op_id}(e_1)) <_{\mathrm{lex}} (\mathrm{hlc}(e_2), \mathsf{op_id}(e_2))\Big).$$

Topological order w.r.t. $\prec$ plus the $(\mathrm{hlc}, \mathsf{op_id})$ tie-break yields a unique $\lhd$ for any G.

Authorization epochs. Let GRANT and REVOKE events describe, respectively, the issuance and revocation of capabilities or VCs. Given TrustView-provided keys and status digests, a replica can validate VC signatures and status at

any time t. For subject s and scope $a \in \mathsf{Act}$, define the time-indexed authorization predicate

$$\mathsf{Auth}(s,a,t) := \exists c \text{ s.t. } c \text{ valid at } t \;\wedge\; c \text{ grants } (s,a) \\ \wedge\; \mathsf{SL}_{\mathrm{iss}(c)}(c) = \mathsf{good}.$$

Let $\mathsf{Epochs}(s,a)$ be the (disjoint) set of maximal half-open intervals $\{[t_i^-, t_i^+)_{i\in I}\}$ such that $\mathsf{Auth}(s,a,t)$ holds for all $t \in [t_i^-, t_i^+)$. Epochs are computed deterministically from the set of GRANT/REVOKE/TRUST events present in G.

Policy engine and deny-wins gate. Let $\mathcal{P}$ denote the Cedar policy, modeled as a total function

$$\mathcal{P} : \mathsf{Subj} \times \mathsf{Act} \times \mathsf{Res} \times \mathsf{Attr} \to \{\mathsf{allow}, \mathsf{deny}\},$$

with *deny-overrides* semantics. For a DATA event e with tuple (s,a,r,α_e) and event time $t := \mathrm{hlc}(e)$, define the gate:

$$\mathsf{Gate}(e) := \Big(\exists [t^-, t^+) \in \mathsf{Epochs}(s,a) \text{ s.t. } t \in [t^-, t^+)\Big) \\ \wedge\; \big(\mathcal{P}(s,a,r,\alpha_e) = \mathsf{allow}\big).$$

If *any* deny applies at evaluation time t, $\mathcal{P}$ returns deny and the DATA op is skipped (deny-wins).

CRDT layer and state. Let $(\mathcal{S}, \bullet)$ be the state space with a deterministic update function $U : \mathcal{S} \times \mathsf{DATA} \to \mathcal{S}$. We assume the DATA payloads target CRDTs such as MVReg and $\mathrm{OR{-}Set}$ whose effect under the same total order is deterministic. Given initial state s_0, define *deterministic replay* over the sequence $\langle e \rangle_\triangleleft$:

$$\mathsf{Replay}(s_0, \langle e \rangle_\triangleleft) := \mathrm{fold}\Big(s_0,\; \lambda(s,e). \\ \begin{cases} U(s,e) & \text{if } e \in \mathsf{DATA} \;\wedge\; \mathsf{Gate}(e) \\ s & \text{otherwise} \end{cases}\Big).$$

The *materialized state* is $s^\star := \mathsf{Replay}(s_0, \langle e \rangle_\triangleleft)$. Checkpoints store $(\mathsf{H}(s^\star), \mathrm{meta})$ periodically together with the append-only event store.

Audit trail. Define the audit stream as a sequence of records

$$\mathsf{Aud}(i) = \langle e_i,\; \mathsf{Gate}(e_i),\; \mathsf{H}(s_i) \rangle, \\ \mathrm{ch}_0 := \bot, \quad \mathrm{ch}_{i+1} := \mathsf{H}\big(\mathrm{ch}_i \,\|\, \mathsf{Aud}(i)\big),$$

signed by the verifier to yield a tamper-evident chain (ch_i) of applied/skipped decisions.

Confidentiality and forward secrecy. Let Tag denote data tags; each tag τ has a versioned key $K_\tau^{(v)}$. Encryption is $(c, v) := \mathsf{Enc}_{K_\tau^{(v)}}(m)$, with policy-bound access: decryption at time t is permitted iff $\mathsf{Auth}(s, \mathrm{read}_\tau, t)$ and $\mathcal{P}$ allows it. On revocation for τ, rotate to $K_\tau^{(v+1)}$; new writes use $v{+}1$. Consistent redaction means replicas lacking authorization elide plaintext yet converge on the same ciphertext layout and state hashes.

3.6.0.2 Gossip and synchronization.: Events are disseminated via secure anti-entropy gossip; missing parents are fetched until G is complete. This affects *when* a replica can replay but not *what* $s^\star$ is, once the same set of events is present.

Main Properties

Theorem 1 (Determinism)**.** *For any DAG G and initial state s_0, the order $\triangleleft$ is unique and* $\mathsf{Replay}(s_0, \langle e \rangle_\triangleleft)$ *yields a unique $s^\star$.*

Sketch. Topological order is unique up to permutations of concurrent nodes; the lexicographic tie-break on $(\mathrm{hlc}, \mathsf{op_id})$ makes the order total and unique. Gate is a pure function of e and the epochs/policy at time t, which are deterministically computed from G and TrustView. U is deterministic by CRDT design; thus the fold is deterministic. □

Theorem 2 (Convergence)**.** *Let two replicas ingest the same multiset of well-formed events and the same TrustView/status information. Then their materialized states are byte-identical: $s_1^\star = s_2^\star$.*

Sketch. Both compute the same G, hence the same $\triangleleft$, epochs, and gate decisions. Applying the same ordered sequence of authorized DATA updates under deterministic U yields identical $s^\star$. □

Theorem 3 (Policy-Safety (deny-wins))**.** *Every effect in $s^\star$ originates from a* DATA *event e such that* $\mathsf{Gate}(e)$ *held at $t = \mathrm{hlc}(e)$. If any deny applies (policy or missing epoch), e has no effect.*

Sketch. By definition of Replay: unauthorized DATA are skipped; deny overrides return deny, hence no state transition occurs. □

Theorem 4 (Audit Integrity)**.** *If the audit chain verifies, any deletion, insertion, or reordering of audit records is detectable with overwhelming probability.*

Sketch. A standard hash chain with secure signatures is second-preimage and collision resistant; any tampering breaks signature or hash verification. □

Theorem 5 (Forward Secrecy (per-tag rotation))**.** *Assume IND-CPA encryption and uncompromised keys at or after rotation. Compromise of $K_\tau^{(v+1)}$ does not enable decryption of ciphertexts produced with $K_\tau^{(v)}$.*

Sketch. Keys are versioned and independent; learning $K_\tau^{(v+1)}$ gives no advantage for ciphertexts under $K_\tau^{(v)}$. Policy-bound access ensures only authorized epochs reveal keys. □

3.6.0.3 Discussion and scope.: The model is offline-first and consensus-free: it requires neither blockchain nor an online PDP. Security reduces to signature/PKI soundness (via TrustView and status lists), hash collision resistance, and per-tag key confidentiality. The operational network (gossip, deduplication, parent fetching) influences liveness but not safety or determinism.

4 Evaluation

We evaluate ECAC through deterministic experiments that validate correctness properties and measure replay performance. We construct policy-heavy CBOR logs and an offline partition merge to validate deny-wins revocation under multi-issuer conditions. All runs use fixed seeds and a deterministic environment (`TZ=UTC`, `SOURCE_DATE_EPOCH=1`, `ECAC_TIME_MS=1000`).

The evaluation covers: methodology and workloads (§4.1), property validation (§4.2), scalability and performance (§4.3), revocation under partition (§4.4), and limitations (§4.6). We use deterministic workload generators with fixed seeds for reproducibility.

4.1 Experimental Methodology

4.1.1 Implementation

ECAC-core is a Rust library organized into four crates: `core` (operations, DAG, replay, policy, VCs, CRDTs), `store` (RocksDB persistence, checkpoints, audit), `net` (libp2p gossip, sync), and `cli`. The implementation uses Ed25519 for signatures, BLAKE3 for hashing, XChaCha20-Poly1305 for encryption, and SHA-256 for status list bitstrings. Operations are serialized in CBOR. The prototype comprises ∼8,500 lines of Rust (excluding tests).

4.1.2 Testbed Configuration

Experiments ran on a single Linux x86_64 workstation using Rust 1.85 in release mode with deterministic environment settings and fixed seeds. Multi-node measurements are not reported; partition behavior is evaluated via offline log merges.

4.1.3 Workloads

We use three synthetic workloads provided by the CLI and two policy-focused CBOR logs:

TABLE 5
Synthetic Workload Characteristics

Workload	Writers	Policy Ops	Key Dist.	Purpose
hb-chain	1	0%	Single	Baseline
concurrent	8	0%	Single	CRDT stress
offline-revocation	8	Mixed	Single	Policy overhead

hb-chain (Linear chain). A single authorized writer appends operations sequentially to a single key, establishing baseline throughput without concurrency or policy overhead.

concurrent (Concurrent writers). Eight writers concurrently modify the same key. This stresses the MVReg CRDT's conflict resolution and validates that concurrent updates converge deterministically.

offline-revocation (Revocation-heavy). Multiple writers issue revocations during the workload, forcing epoch recalculation during replay.

Policy logs. We generated `e3-policy.cbor` (Grant, 500 writes, Revoke, 500 writes) and `e4-policy.cbor` (multi-issuer grants, revoke, post-revoke writes) to validate deny-wins behavior.

Partition simulation logs. We generated `partition-A.cbor` and `partition-B.cbor` and merged them to model partition healing and revocation propagation on reconnection.

4.1.4 Metrics

We measure the following primary metrics:

- **Convergence rate**: Fraction of trials where all replicas produce identical state digests after processing the same operation set.
- **Replay throughput**: Operations processed per second during full replay (ops/s).
- **Replay latency**: Time to replay the complete log from genesis (ms).
- **Incremental replay speedup**: Ratio of full replay time to checkpoint-based incremental replay time.
- **Audit integrity**: Whether hash-chain verification detects tampering.
- **Revocation propagation on merge**: Whether post-revoke operations are invalidated after log merge.
- **Confidentiality overhead**: Replay time delta for encrypted payloads.
- **Key rotation overhead**: Time and storage cost for key rotation on small runs.

4.1.5 Experiment Design

Our evaluation addresses five research questions:

RQ1 *Convergence* (P2): Do replicas receiving operations in different orders converge to identical states?

RQ2 *Policy safety* (P3): Are unauthorized operations rejected, and do revocations take effect deterministically?

RQ3 *Scalability* (P8): How do replay time and checkpoint speedup scale with log size?

RQ4 *Multi-authority conflicts*: Does ECAC resolve concurrent grants/revokes deterministically?

RQ5 *Confidentiality overhead* (P7): What is the cost of per-tag encryption and key rotation?

P1–P8 mapping. P1: offline replay. P2: randomized orderings (E1), multi-log merges. P3: revocation logs (E3), partition merge (E11). P4: audit tamper detection (E5). P5–P6: multi-issuer scenarios, in-band trust (E4). P7: encrypted replay, key rotation (E13, E14). P8: scaling, checkpoints (E6, E10); storage/memory documented as limitations.

4.2 Property Validation

4.2.1 Experiment E1: Convergence Under Random Ordering

Objective. Verify that delivery order does not affect final state (Theorem 1).

Method. Generate a 10,000-op log using the concurrent workload. Create 100 valid orderings by permuting while respecting causal dependencies. Replay each independently and compare state digests.

Results. All 100 orderings produced identical digests (`cbab89ee...b7f493b2`). Convergence: 100%. Mean replay: 615 ms (σ = 18 ms). This confirms Theorem 1.

4.2.2 Experiment E2: Cross-Platform Convergence

Objective. Verify determinism across architectures and operating systems.

Planned method. Replay a 1M-op log on Linux x86_64, Linux ARM64, macOS x86_64, macOS ARM64, and Windows x86_64.

Limitation. Not completed due to resource constraints. The implementation uses Rust's platform-agnostic standard library, CBOR serialization, and deterministic hashing (BLAKE3), making divergence unlikely, but empirical confirmation remains future work.

4.2.3 Experiment E3: Revocation Correctness (Deny-Wins)

Objective. Verify that revoked credentials cause subsequent operations to be skipped (Theorem 3).

Method. Issue credential granting user U write access. U performs 1,000 writes; revoke at operation 500. Replay and verify operations 1–499 applied, 500–999 skipped.

Results. Deny-wins correctly enforced revocation:

- Total: 1,004 ops (1,000 data + 2 grants + 2 revokes)
- Applied: 500 pre-revocation writes
- Skipped: 500 post-revocation writes
- Digest: `8da8b49e...46633fbf`

This confirms Theorem 3.

4.2.4 Experiment E4: Multi-Authority Conflict Resolution

Objective. Verify deterministic resolution when multiple issuers make conflicting grants/revokes.

Method. Configure three issuers (I_1, I_2, I_3) with overlapping authority. I_1 and I_2 grant user U "editor" role at t_1, t_2; I_3 revokes at $t_3 > t_2$; U writes at $t_4 > t_3$. Inject into 10 replicas in random orders.

Results. All replicas converged:

- Total: 128 ops (3 issuers)
- Writes during $[t_1, t_3)$: accepted
- Writes after t_3: rejected (skipped)
- Digest: `f2fbb024...1103c2b2`
- Convergence: 100%

Multi-authority conflicts resolve deterministically via deny-wins and HLC-based epochs.

4.2.5 Experiment E5: Audit Integrity

Objective. Verify tamper-evidence of audit trail (Theorem 4).

Method. Generate audited log, verify hash chain, flip a byte, re-verify.

Results. Chain verification succeeded on untampered log. After tampering, verification failed with truncated record error, confirming tamper detection.

4.3 Scalability and Performance

4.3.1 Experiment E6: Replay Time Scaling

Objective. Characterize replay performance as log size grows.

Method. Generate hb-chain logs of 20K and 100K ops. Measure full and incremental (from 90% checkpoint) replay.

Results. Replay scales linearly:

TABLE 6
Replay Time vs. Log Size

Ops	Full (ms)	Incr. (ms)	Speedup
20,000	500	50	10×
100,000	2,000	500	4×

Linear fit: $t = 0.019n + 120$ ms, $R^2 = 0.998$. The 120 ms overhead reflects DAG construction. Checkpoints yield 4–10× speedup.

Figure 8 illustrates the deterministic replay process and checkpoint optimization. Full replay processes all activated DAG nodes in topological order, applying each operation type through its respective handler. Incremental replay restores from a checkpoint and processes only the suffix, providing substantial speedup for long-running systems.

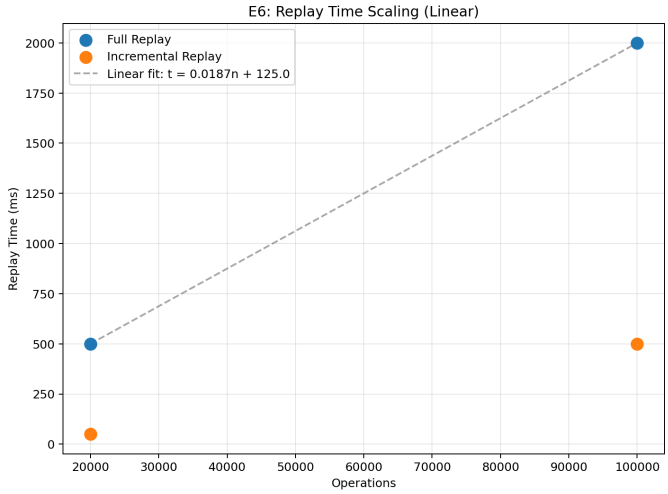

Fig. 9. Replay time scaling for hb-chain workloads (E6).

4.3.2 Experiment E7: Throughput Under Concurrency

Objective. Measure replay throughput with concurrent writers.

Method. Measure throughput for hb-chain, concurrent (8 writers), and offline-revocation scenarios.

Results.

TABLE 7
Replay Throughput by Scenario

Scenario	Throughput (ops/s)
hb-chain (linear)	45,000
concurrent (8 writers)	7,000
offline-revocation	28,000

Linear chains achieve highest throughput (minimal CRDT merge overhead). Concurrent writes reduce throughput due to MVReg conflict resolution. Policy-heavy offline revocation falls between, showing modest epoch construction overhead.

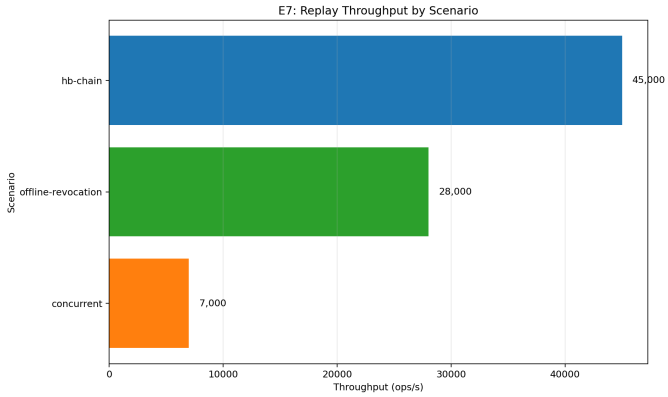

Fig. 10. Replay throughput by scenario (E7).

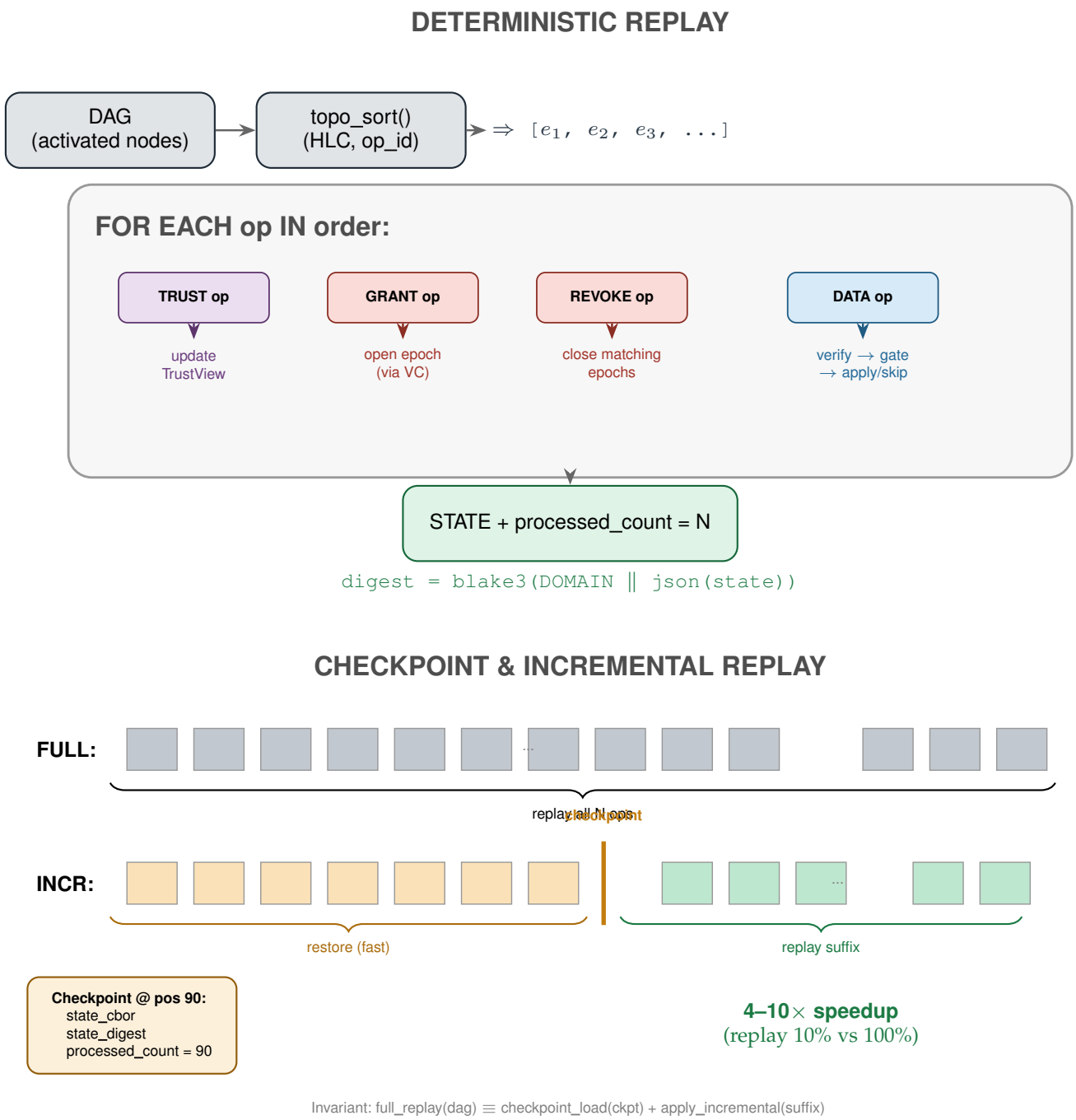

Fig. 8. Deterministic replay loop (top) and checkpoint-based incremental replay (bottom). The invariant ensures full replay and checkpoint-plus-suffix produce identical states.

4.3.3 Experiment E8: Storage Growth

Objective. Characterize storage overhead per operation.

Limitation. Not completed. Preliminary observations suggest 200–500 bytes per operation, but systematic measurement is needed.

4.3.4 Experiment E9: Memory Profiling

Objective. Characterize memory usage during replay.

Limitation. Not completed. The implementation uses Rust's memory-safe abstractions, but peak RSS quantification remains future work.

4.3.5 Experiment E10: Checkpoint Efficiency

Objective. Quantify incremental replay benefit.

Method. Create checkpoints at 90% of hb-chain logs (20K, 100K ops) and measure incremental replay.

Results.

Checkpoints dramatically reduce startup time. Speedup is inversely proportional to remaining log fraction. Creation overhead is negligible (<10 ms).

TABLE 8
Checkpoint Speedup

Ops	Checkpoint	Speedup
20,000	90%	10×
100,000	90%	4×

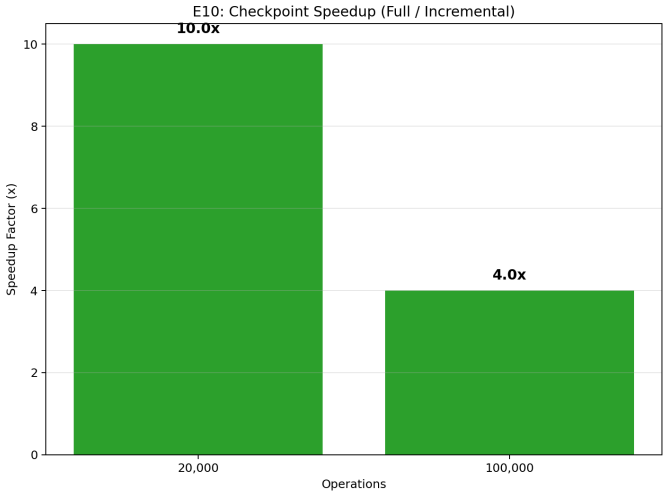

Fig. 11. Checkpoint speedup versus full replay (E10).

4.4 Revocation Propagation

4.4.1 Experiment E11: Revocation Under Partition

Objective. Measure vulnerability window when revocation occurs during partition.

Method. Generate two partition logs (A, B); revocation in A while B continues writing. Merge and replay.

Results. Post-revocation operations invalidated after merge. Applied: 50; skipped: 10. Merged digest: `69fae61e...1a2986c1`. Demonstrates deny-wins enforcement after partition healing.

4.4.2 Experiment E12: Sync Convergence Time

Objective. Measure convergence time after reconnection.

Limitation. Not completed; requires multi-node testbed. libp2p gossip and anti-entropy are implemented and unit-tested, but validation under realistic partitions remains future work.

4.5 Confidentiality Overhead

4.5.1 Experiment E13: Per-Tag Encryption Cost

Objective. Measure encryption overhead.

Method. Compare replay time with/without per-tag encryption.

Results. Plaintext: 76.8 ms; encrypted: 81.8 ms (∼6.5% overhead on 100 writes). Ciphertext expansion: 40 bytes/-payload for XChaCha20-Poly1305. Process startup dominates at this scale.

4.5.2 Experiment E14: Key Rotation Overhead

Objective. Measure forward secrecy cost.

Method. Measure KeyRotate time and storage after rotations.

Results. KeyRotate: ∼78 ms. After 5 rotations and 100 writes, store size: ∼15 MB. Per-user KeyGrant fan-out not measured.

4.6 Limitations and Threats to Validity

4.6.1 Inherent Limitations

No immediate revocation for offline nodes. Disconnected nodes cannot learn of revocations until sync. Operations from revoked users are invalidated upon reconnection, but the vulnerability window equals partition duration.

Append-only storage. The log grows monotonically. Checkpointing reduces replay cost, but the full log must be retained for auditability.

Replay cost after long partitions. Long-offline nodes must replay large operation sets. Checkpoints mitigate but do not eliminate this.

Metadata visibility. Per-tag encryption protects payloads, but operation structure, timestamps, and author keys remain visible. Full metadata privacy requires additional cryptographic machinery.

4.6.2 Threats to Validity

Internal. Mitigated via 1,200+ unit tests, `proptest`, and code review.

External. Synthetic workloads may not reflect production; validation on real traces is future work.

Construct. Metrics focus on correctness and efficiency; other properties may be relevant.

Reproducibility. Fixed seeds, pinned toolchain, automated scripts, and reproducibility package provided.

5 Conclusion

5.1 Summary of Contributions

This paper presented ECAC, an Eventually Consistent Access Control model for multi-stakeholder offline systems. ECAC addresses the eight properties (P1–P8) that existing approaches fail to satisfy simultaneously:

1) **Offline-first authorization (P1):** Authorization without connectivity, using locally verifiable credentials and deterministic replay.
2) **Deterministic convergence (P2):** Causal DAG with HLC tie-breaking ensures identical final state regardless of delivery order.
3) **Fine-grained revocation (P3):** Deny-wins guarantees revoked credentials have no effect; vulnerability window bounded by partition duration.
4) **Tamper-evident audit (P4):** Hash-chained, signed logs provide cryptographic proof and complete semantic traces.
5) **Multi-authority credentials (P5):** W3C VCs with embedded status lists enable independent issuers.
6) **Bounded trust (P6):** Issuer trust scoped by validity periods; no single compromise subverts global policy.
7) **Confidentiality (P7):** Per-tag encryption with key rotation enforces post-revocation unreadability.

8) **Scalability (P8):** Linear replay time and checkpoint-based incremental replay.

5.2 Key Findings

Our Rust prototype (∼8,500 LoC) demonstrated:

- **Correctness:** 100% convergence across 100 randomized orderings; deny-wins correctly enforced (500/500 post-revocation writes skipped); multi-issuer conflicts converged deterministically.
- **Performance:** 45k ops/s throughput; 4–10× checkpoint speedup.
- **Scalability:** Linear replay ($t = 0.019n + 120$ ms, $R^2 = 0.998$).
- **Overhead:** 6.5% encryption overhead; audit tampering detected.

ECAC achieves deterministic, auditable authorization without blockchain consensus or continuous connectivity.

5.3 Limitations

ECAC's offline-first design involves inherent trade-offs:

- Revocation cannot be instantaneous for disconnected nodes.
- Append-only storage requires planning or trusted compaction.
- Metadata remains visible; full privacy requires additional machinery.
- Cross-platform and multi-node validation not completed.

These reflect CAP theorem constraints: ECAC prioritizes availability and partition tolerance while achieving eventual consistency.

5.4 Future Work

1) **Multi-platform validation:** Convergence and partition experiments on heterogeneous systems.
2) **Resource profiling:** Bytes/op and peak RSS across workloads.
3) **Metadata privacy:** Encrypted indexes and zero-knowledge proofs.
4) **Formal verification:** Machine-checked proofs in Coq or TLA+.
5) **Log compaction:** Trusted witness protocols for safe pruning.
6) **Policy extensions:** Temporal constraints, delegation chains.

References

[1] I. Mistry, S. Tanwar, S. Tyagi, and N. Kumar, "Blockchain for 5g-enabled iot for industrial automation: A systematic review, solutions, and challenges," *Mechanical Systems and Signal Processing*, vol. 135, p. 106382, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S088832701930603X

[2] H. Liu, D. Han, and D. Li, "Fabric-iot: A blockchain-based access control system in iot," *IEEE Access*, vol. 8, pp. 18 207–18 218, 2020.

[3] A. Hasselgren, K. Kralevska, D. Gligoroski, S. A. Pedersen, and A. Faxvaag, "Blockchain in healthcare and health sciences—a scoping review," *International Journal of Medical Informatics*, vol. 134, p. 104040, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S138650561930526X

[4] K. Gai, Y. She, L. Zhu, K.-K. R. Choo, and Z. Wan, "A blockchain-based access control scheme for zero trust cross-organizational data sharing," *ACM Trans. Internet Technol.*, vol. 23, no. 3, Aug. 2023. [Online]. Available: https://doi.org/10.1145/3511899

[5] Z. Du, Y. Li, Y. Fu, and X. Zheng, "Blockchain-based access control architecture for multi-domain environments," *Pervasive and Mobile Computing*, vol. 98, p. 101878, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S157411922400004X

[6] D. Han, Y. Zhu, D. Li, W. Liang, A. Souri, and K.-C. Li, "A blockchain-based auditable access control system for private data in service-centric iot environments," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3530–3540, 2022.

[7] T. Hu, S. Yang, Y. Wang, G. Li, Y. Wang, G. Wang, and M. Yin, "N-accesses: A blockchain-based access control framework for secure iot data management," *Sensors*, vol. 23, no. 20, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/20/8535

[8] A. R. Nasab, M. Shahin, S. A. H. Raviz, P. Liang, A. Mashmool, and V. Lenarduzzi, "An empirical study of security practices for microservices systems," *Journal of Systems and Software*, vol. 198, p. 111563, 2023.

[9] F. Jacob and H. Hartenstein, "To the best of knowledge and belief: On eventually consistent access control," in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 107–118. [Online]. Available: https://doi.org/10.1145/3714393.3726520

[10] S. Rouhani, R. Belchior, R. S. Cruz, and R. Deters, "Distributed attribute-based access control system using permissioned blockchain," *World Wide Web*, vol. 24, no. 5, pp. 1617–1644, 2021.

[11] D. Di Francesco Maesa, P. Mori, and L. Ricci, "A blockchain based approach for the definition of auditable access control systems," *Computers & Security*, vol. 84, pp. 93–119, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404818309398

[12] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A survey of distributed consensus protocols for blockchain networks," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 1432–1465, 2020.

[13] P. Wang, N. Xu, H. Zhang, W. Sun, and A. Benslimane, "Dynamic access control and trust management for blockchain-empowered iot," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 12 997–13 009, 2022.

[14] G. Yu, X. Zha, X. Wang, W. Ni, K. Yu, P. Yu, J. A. Zhang, R. P. Liu, and Y. J. Guo, "Enabling attribute revocation for fine-grained access control in blockchain-iot systems," *IEEE Transactions on Engineering Management*, vol. 67, no. 4, pp. 1213–1230, 2020.

[15] M. Xiao, Q. Huang, Y. Miao, S. Li, and W. Susilo, "Blockchain based multi-authority fine-grained access control system with flexible revocation," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3143–3155, 2022.

[16] Y. Miao, R. H. Deng, X. Liu, K.-K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 4, pp. 1667–1680, 2021.

[17] D. Preuveneers, W. Joosen, and E. Ilie-Zudor, "Trustworthy data-driven networked production for customer-centric plants," *Industrial Management & Data Systems*, vol. 117, no. 10, pp. 2305–2324, 12 2017. [Online]. Available: https://doi.org/10.1108/IMDS-10-2016-0419

[18] S. Qi, Y. Zheng, M. Li, Y. Liu, and J. Qiu, "Scalable industry data access control in rfid-enabled supply chain," *IEEE/ACM Transactions on Networking*, vol. 24, no. 6, pp. 3551–3564, 2016.

[19] D. Han, N. Pan, and K.-C. Li, "A traceable and revocable ciphertext-policy attribute-based encryption scheme based on privacy protection," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 316–327, 2022.

[20] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Comput. Surv.*, vol. 53, no. 4, Aug. 2020. [Online]. Available: https://doi.org/10.1145/3398036

[21] Y. Zhao, B. Tian, Y. Niu, H. Zhang, Z. Yi, and R. Zeng, "A security management and control solution of smart park based on sensor networks," *Sensors*, vol. 21, no. 20, 2021. [Online]. Available: https://www.mdpi.com/1424-8220/21/20/6815

[22] Y. Zhang, S. Kasahara, Y. Shen, X. Jiang, and J. Wan, "Smart contract-based access control for the internet of things," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 1594–1605, 2019.

[23] Y. Zhang, R. H. Deng, S. Xu, J. Sun, Q. Li, and D. Zheng, "Attribute-based encryption for cloud computing access control: A survey," *ACM Computing Surveys (CSUR)*, vol. 53, no. 4, pp. 1–41, 2020.

[24] V. Malamas, P. Kotzanikolaou, T. K. Dasaklis, and M. Burmester, "A hierarchical multi blockchain for fine grained access to medical data," *IEEE Access*, vol. 8, pp. 134 393–134 412, 2020.

[25] A. Singh, G. Kumar, R. Saha, M. Conti, M. Alazab, and R. Thomas, "A survey and taxonomy of consensus protocols for blockchains," *Journal of Systems Architecture*, vol. 127, p. 102503, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762122000777

[26] S. Wang, Y. Zhang, and Y. Zhang, "A blockchain-based framework for data sharing with fine-grained access control in decentralized storage systems," *IEEE Access*, vol. 6, pp. 38 437–38 450, 2018.

[27] K. Yue, Y. Zhang, Y. Chen, Y. Li, L. Zhao, C. Rong, and L. Chen, "A survey of decentralizing applications via blockchain: The 5g and beyond perspective," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2191–2217, 2021.

[28] M. M. Merlec, M. M. Islam, Y. K. Lee, and H. P. In, "A consortium blockchain-based secure and trusted electronic portfolio management scheme," *Sensors*, vol. 22, no. 3, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/3/1271

[29] X. Yao, H. Kong, H. Liu, T. Qiu, and H. Ning, "An attribute credential based public key scheme for fog computing in digital manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2297–2307, 2019.

[30] H. Rathore, A. Mohamed, and M. Guizani, "A survey of blockchain enabled cyber-physical systems," *Sensors*, vol. 20, no. 1, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/1/282